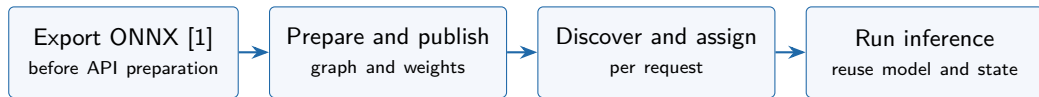


1. From a Model File to Distributed Inference



```
auto model = user.prepare("qwen");  
auto call = user.request(model, input, options); // one invocation  
// For multiple turns:  
auto chat = user.openConversation(model);  
auto turn = chat.request(input, options);
```

Choose a single invocation or a multi-turn conversation. The User submits the work; `model` is a handle to prepared model data, not a running model.

Key point

Preparing model data and choosing where to execute are separate steps. A prepared model can support later requests.

Evidence: User-approved API revision; implementation updated separately. [1] ONNX documentation. Configuration and result handling omitted.

2. prepare(): Validate and Publish Model Materials

1. **Before prepare:** export trained model files to ONNX and configure the model description and model-specific processing code.
2. **Read and validate:** check graph/weight hashes, supported operations, input/output tensor formats, and size limits.
3. **Publish retrievable layers:** store the graph, weights, and layer indexes in a Repo—a named-data storage service. Each assigned layer can then be retrieved by reference.
4. **Return a prepared handle:** retain model references and validation metadata for later requests, without starting inference.

Key point

Preparing retrievable layers does not assign them to Providers. Assignment happens later, using Provider feedback.

Preparation cache: reuse a ready entry or share an in-progress preparation. Active handles keep the entry in use; configured limits control retention and refresh.

Evidence: Runtime.cpp: source loader and User::prepare; ModelPreparationCache.cpp; NativeRequestCatalog.cpp.

3. request(): Use Provider Feedback to Assign Layers



- ▶ The User specifies the task, deadline, generation budget, and model/input references—not the full model in every Request.
- ▶ Providers return acknowledgment (ACK) feedback. A positive ACK indicates willingness; the User also checks eligibility.
- ▶ After ACK collection closes, the User's placement strategy decides which Provider executes each layer group and which intermediate outputs it needs.
- ▶ Selection tells each chosen Provider what to execute and supplies the protected references and authorization needed for that work.

Key point

A positive ACK offers participation. Only a validated Selection permits the assigned execution; an ACK does not reserve resources.

Evidence: PreparedModel.cpp; native planning/Selection path; Spec189 r259 affinity evidence.

4. After Selection: Reuse, Fetch, Assemble, Load

1. **Verify the assignment:** validate authorization, model identity, selected role, and permitted layer/material references.
2. **Check local model data:** reuse files only when their model version, assigned layers, authorization, and validity match this request.
3. **On a miss, retrieve assigned materials:** express Interests for the required Named Data; verify and decrypt the retrieved materials.
4. **Assemble a model fragment:** combine the assigned layers and weights; define tensor inputs from preceding layers and outputs for subsequent layers.
5. **Load the fragment:** create an ONNX Runtime session. This loaded session and its execution wrapper are called the **runner**.

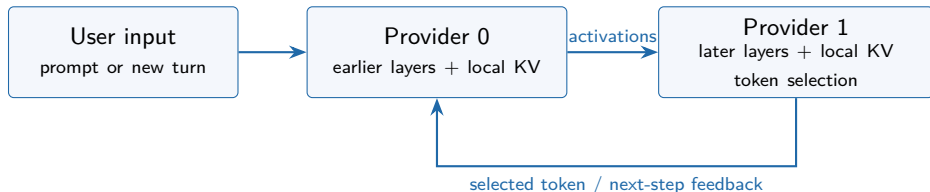
Key point

Having model files on disk does not mean the model is loaded in memory. Loading can still be required.

Validation limit: the completed three-turn test used a cache-compatible path; it did not validate the full encrypted-Repo retrieval and reuse path.

Evidence: Provider.cpp; ProviderArtifactCache.cpp; NativeCanonicalOnnxAssembler.cpp; Spec190 T003/T006.

5. Prefill and Decode Across Two Providers



- ▶ **Prefill:** process input tokens and compute attention keys and values (KV). Each Provider retains KV for its own layers.
- ▶ **Decode:** use the previous token and cached KV to compute the next token. Activations are intermediate tensors passed between model fragments.
- ▶ Arrows show logical dependencies: each consumer retrieves and validates Named Data before its next step; Data is not unsolicited push.
- ▶ The last Provider publishes generated tokens for the User. End-of-sequence (EOS) or the token limit stops generation; finalization saves continuation state.

Evidence: NativeEpochCoordinator.cpp; ONNX model runner; Spec189 r259/r260 generation evidence. KV = attention keys/values.

6. Different Caches Avoid Different Work

Cache / owner	What it retains	What a hit can avoid
Preparation / User	Package and publication references	Repeated preparation/publication
Model data / Provider	Model references and reusable files	Retrieval or assembly when reuse checks pass
Runner / Provider	Loaded model and runtime session	Loading at each generation step
KV / Provider	Attention keys/values for processed tokens	Repeating attention-state computation for those tokens
Content Store / NDN forwarder	Previously retrieved Data packets	Retrieval from the original serving node, if cached

Key point

Model files, loaded runners, and KV are different caches. All reuse requires applicable validity checks; full encrypted-Repo reuse remains under validation.

Evidence: `ModelPreparationCache.cpp`; `ProviderArtifactCache.cpp`; `NativeEpochCoordinator.cpp`; Spec189 r260.

7. Restore KV and Continue the Next Turn

1. **Save progress:** each Provider retains its layers' attention keys/values (KV) for a limited time. A checkpoint identifies the processed token prefix and references the retained state.
2. **Keep a continuation record:** the conversation journal stores authenticated checkpoint and Provider references, not the full KV tensors.
3. **Choose Providers again:** prefer previous Providers if still authorized and suitable; this preference does not reserve them.
4. **Restore compatible KV:** verify the model, assigned layers, token history, retained state, and expiry before reuse.
5. **Continue inference:** process the new input tokens, then generate output using the restored and newly extended KV.

Key point

Restored KV saves prefix computation. The model runner must also be ready; keeping KV alone does not eliminate session loading.

If retained KV is unavailable or invalid, report failure unless a supported path can rebuild it by processing the token history again.

Evidence: `Conversation.cpp`; `Spec189 r259 retention/affinity` and `r260 KV_RESTORED` evidence.

8. Keep the Model Loaded—Within Explicit Limits

Verified: within one request

One loaded runner per selected role is reused across generation steps and finalization **within a request**.

Three turns on two Providers required **6 runner preparations**: one per Provider per turn, not one per generated token.

Target: reuse across turns

Retain an idle loaded session when memory budget and lifetime permit; reuse it for a compatible, newly authorized request.

Limit total retained memory and idle time. Keep active runners alive, release unused entries when needed, and check authorization for every new request.

Key point

When memory is available, keeping a compatible runner can avoid loading it again. Retaining the runner does not automatically retain or validate a conversation's KV.

Keeping a loaded runner across requests is a target, not a result established by the three-turn test.

Evidence: NativeEpochCoordinator.cpp: request-local runner slot; Spec189 r260; Spec190 T007.

9. Where the Large Resource Costs Occur

Operation	Main cost drivers	Reuse opportunity
ONNX export / preparation	Graph processing, weight reads, temporary memory	Export once; retain prepared package
Repo publication	Encryption, hashing, storage writes, transferred bytes	Reuse immutable published materials
Provider material retrieval	Network transfer, validation, decryption	Fetch only assigned layers; eligible local hits
Assembly / session loading	Graph and weight copies, disk I/O, runtime allocations	Reuse assembled artifacts and loaded sessions
Prefill / decode	Compute, growing KV memory, activations each step	Reuse prefix KV; avoid per-token runner loading

Key point

Encryption/decryption can temporarily retain several full-model buffers, not just a key. These buffers may overlap with assembly and loaded runners on the same host.

Evidence: Spec190 T003 resource evidence. Cost drivers are structural, not a measured ranking; stage-level timing/bytes remain needed.

10. Results: Two Providers, Three Conversation Turns

MiniNDN [2], two Provider nodes: three turns completed with real ONNX inference and KV restoration in turns 2–3. This test used reusable model data; full encrypted-Repo retrieval was not evaluated.

Turn	Prior KV tokens	New input tokens	Output tokens*	Saved KV tokens
1	0	13	10	23
2	23	17	11	51
3	51	17	11	79

Turn 2: $23 + 17 + 11 = 51$. Saved KV covers prior context, new input, and generated output; it is a token count, not a memory size.

6 runner preparations: one per Provider per turn. Total time from Selection to saved checkpoint across three turns: **63.01 s**, including preparation, computation, transfer, and checkpointing.

Key point

Multi-turn generation and KV reuse succeeded. The 1024-token limit was a budget; EOS ended each turn earlier. Full protected-Repo and sustained long-output tests remain open.

*Output counts include EOS; all turns stopped at EOS. One run; journals passed between requester processes. Clean shutdown.

Evidence: Local execution evidence and audit trail: [review-20260922.md](#). [2] Mini-NDN project.